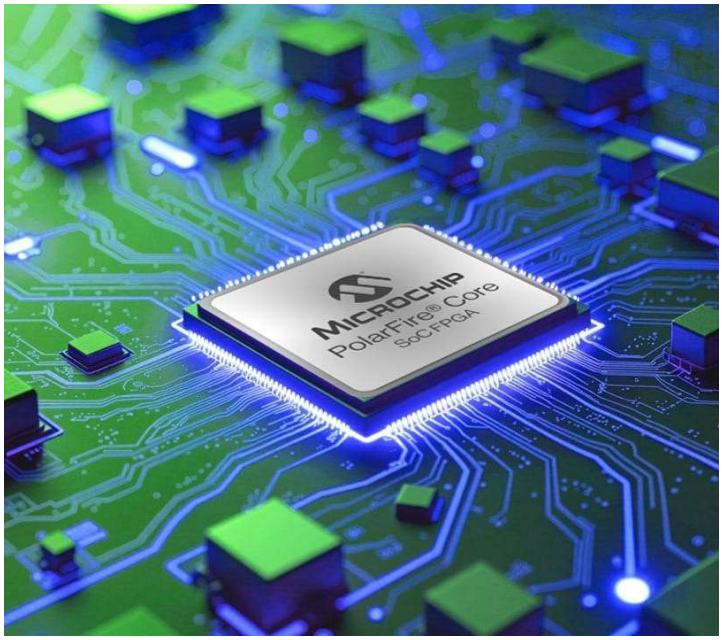
DIGITAL SYSTEMS LAB

7-SEGMENT DISPLAY DRIVER

Lab 01



OBJECTIVE

Design and implementation of a driver for four 7-segment LED displays, enabling the cyclic scrolling of a message on the Boolean Board featuring a Xilinx Spartan-7 XC7S50-CSGA324 FPGA.

ANASTASIA EFTHYMIOU – ANDROMACHE SPYROU

03988 – 04059

Friday, 7 November 2025

Contents

OBJECTIVE 1

INTRODUCTION: LABORATORY ASSIGNMENT 4

Objectives 4

Summary 4

PART A: Implementation of the 7-Segment Decoder 5

Implementation 5

Verification 5

PART B: Driving Four Digits 6

Implementation 6

Top-level module overview 6

MMCM module 6

counter module 7

Implementation 7

AnodeDriving module 7

Implementation 7

KathodeDriving module 7

Implementation 7

Verification 8

Experiment and Final Verification 8

PART C: Stepwise Message Rotation Using a Pushbutton 9

Implementation 9

Top-level module overview 9

reset_synchr module 9

Implementation 9

button_deb module 9

Implementation 9

KathodeDriving2 module 10

Implementation 10

MessageRotator module 10

Implementation 10

Verification 10

Experiment and Final Verification 10

PART D: Stepwise Message Rotation with a Fixed Delay 11

Implementation 11

Top-level module overview 11

counter_D module 11

Implementation 11

pointer module 11
Implementation 11
Verification 11
Experiment and Final Verification 12
CONCLUSIONS: Summary of the Work 12

INTRODUCTION: LABORATORY ASSIGNMENT

Objectives

This assignment concerns the development of a driver for four 7-segment LED displays and the cyclic presentation of a 16-character message. More specifically, the task involves designing a 7-segment decoder, appropriately driving the four digits and displaying a static message, and implementing stepwise rotation of the displayed message either by pressing a pushbutton or after a specified time interval.

Summary

The work was carried out using the Boolean Board with a Xilinx Spartan-7 XC7S50-CSGA324 FPGA, the Verilog hardware description language to describe the interconnections and behavior of the system and to simulate its operation, and the Vivado software suite for system design, implementation, and functional verification.

A 7-segment decoder was designed and functionally verified to convert 4-bit binary inputs into the appropriate LED-driving signals. A four-digit control circuit was also implemented using alternating anode activation and timing provided by an MMCM module. Subsequently, a circuit was developed to store a 16-character message and shift it one step at a time, initially under pushbutton control. In the final stage, the shifting process was automated using a clock counter to provide a constant delay.

The system comprises the following modules, which are described in detail in the subsequent sections:

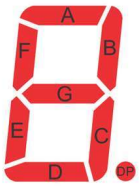
- LEDdecoder – decodes a character for presentation on the board LEDs.
- counter – determines when the anodes and cathodes are activated.
- AnodeDriving – controls the anodes.
- KathodeDriving – controls the cathodes.
- KathodeDriving2 – controls the cathodes for a changing message.
- reset_synchr – synchronizes the reset signal.
- button_deb – debounces the pushbutton.
- MessageRotator – initializes the message and rotates it under pushbutton control.
- address – advances the message address in response to the pushbutton.
- counter_D – provides the counter used for automatic message rotation.
- pointer – provides the memory pointer used for automatic message rotation.
- FourDigitLEDdriver – instantiates the MMCM and interconnects the modules.

In summary, the system displays four characters of a message on the 7-segment displays at a time, with appropriate timing and synchronization. The message shifts one position to the right and repeats cyclically.

PART A: Implementation of the 7-Segment Decoder

Implementation

A 7-segment display driver requires a combinational decoder to represent the hexadecimal values to be displayed. The decoder receives a 4-bit binary value (char) and produces a 7-bit output (LED), whose bits correspond to the signals driving the seven segments of the display. Each output bit controls one segment, allowing hexadecimal digits to be represented.



7-segment display and segment labels.

A combinational Verilog module, LEDdecoder, was developed to assign the appropriate output values to LED according to the input char. To illuminate a particular segment, both the associated anode signal and the corresponding segment cathode signal must be driven low (0). This behavior is implemented through an always block sensitive to char, containing a case statement with 16 cases to select the output values.

Verification

A dedicated testbench was developed and used to verify correct operation for every possible input value.

```
char=0|LED = 0000001|Kathodes open: a = 0,b = 0,c = 0,d = 0,e = 0,f = 0,g = 1
char=1|LED = 1001111|Kathodes open: a = 1,b = 0,c = 0,d = 1,e = 1,f = 1,g = 1
char=2|LED = 0010010|Kathodes open: a = 0,b = 0,c = 1,d = 0,e = 0,f = 1,g = 0
char=3|LED = 0000110|Kathodes open: a = 0,b = 0,c = 0,d = 0,e = 1,f = 1,g = 0
char=4|LED = 1001100|Kathodes open: a = 1,b = 0,c = 0,d = 1,e = 1,f = 0,g = 0
char=5|LED = 0100100|Kathodes open: a = 0,b = 1,c = 0,d = 0,e = 1,f = 0,g = 0
char=6|LED = 0100000|Kathodes open: a = 0,b = 1,c = 0,d = 0,e = 0,f = 0,g = 0
char=7|LED = 0001111|Kathodes open: a = 0,b = 0,c = 0,d = 1,e = 1,f = 1,g = 1
```

Decoder testbench results, input values 0–7.

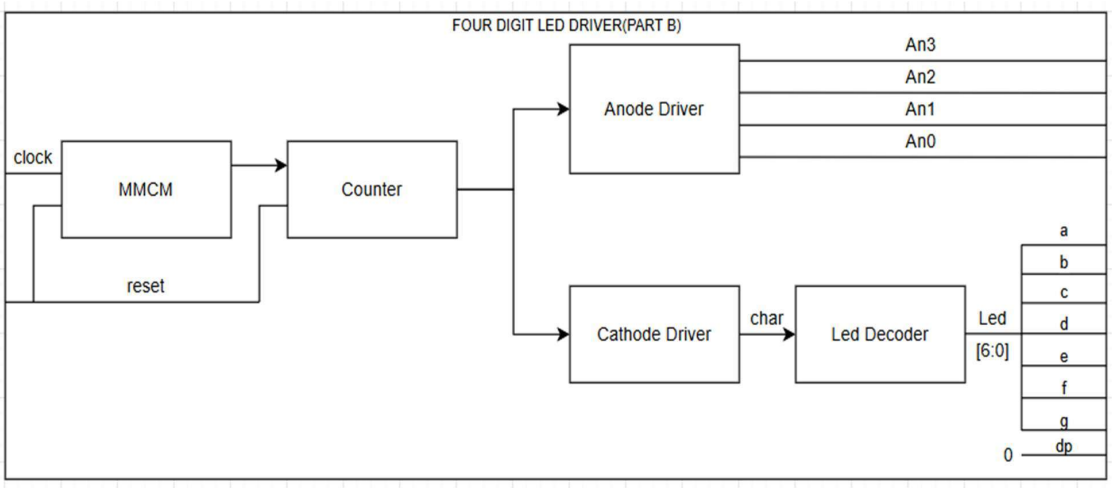
```
char=8|LED = 0000000|Kathodes open: a = 0,b = 0,c = 0,d = 0,e = 0,f = 0,g = 0
char=9|LED = 0000100|Kathodes open: a = 0,b = 0,c = 0,d = 0,e = 1,f = 0,g = 0
char=a|LED = 0001000|Kathodes open: a = 0,b = 0,c = 0,d = 1,e = 0,f = 0,g = 0
char=b|LED = 1100000|Kathodes open: a = 1,b = 1,c = 0,d = 0,e = 0,f = 0,g = 0
char=c|LED = 0110001|Kathodes open: a = 0,b = 1,c = 1,d = 0,e = 0,f = 0,g = 1
char=d|LED = 1000010|Kathodes open: a = 1,b = 0,c = 0,d = 0,e = 0,f = 1,g = 0
char=e|LED = 0110000|Kathodes open: a = 0,b = 1,c = 1,d = 0,e = 0,f = 0,g = 0
char=f|LED = 0111000|Kathodes open: a = 0,b = 1,c = 1,d = 1,e = 0,f = 0,g = 0
```

Decoder testbench results, input values 8–F.

PART B: Driving Four Digits

Implementation

Top-level module overview



Top-level architecture for the four-digit driver (Part B).

The top-level module comprises the following components:

- LEDdecoder (from Part A)
- counter
- AnodeDriving
- KathodeDriving
- MMCM

MMCM module

The MMCM (Mixed-Mode Clock Manager) is an important subsystem in modern FPGA devices. It is designed to generate and manage clock signals, including their phase, frequency, and delay. The MMCM receives a clock input (CLKIN) and a reset input (RST), and can generate up to seven output clock signals. Only one of these outputs is used in this assignment. The MMCM can multiply or divide the incoming clock frequency within prescribed operating limits to ensure system stability and proper operation.

To alternate the driving of the digits while providing a minimum charging delay of 0.20 µs, a clock with a period 20 times longer than that of the FPGA board clock is required. The following equations were used to determine the parameter values:

$$F_{VCO} = (F_{IN} \times M) / D \qquad F_{OUT} = (F_{IN} \times M) / (D \times O)$$

The parameter settings were checked against the following operating limits:

- F_IN – input frequency (CLKIN): 10–800 MHz
- D – input divider (DIVCLK_DIVIDE): 1–106
- M – feedback multiplier (CLKFBOUT_MULT_F): 2–64
- O – output divider (CLKOUT_DIVIDE): 1–128
- F_VCO – VCO oscillation frequency: 4.69–800 MHz

After several iterations of adjusting and checking the MMCM parameters, the desired 200 ns output clock period was obtained, within the limits specified by the system technical specifications, using the following values:

- F_IN = 100 MHz
- D = 120

- M = 6
- O = 1

counter module

The counter module is a sequential circuit used to ensure non-overlapping anode and cathode driving on the board. Its inputs are clock and reset, and its output represents one of 16 states. The counter starts at 1111 and decrements by one on every rising edge of the slow clock generated by the MMCM, continuing cyclically until it reaches 0000.

Implementation

The module contains an always block with posedge clock and posedge reset in its sensitivity list. Through an if–else condition, the counter is set to 1111 while reset is asserted; otherwise, it decrements by one.

AnodeDriving module

The anode-driving module uses the counter state to activate the appropriate anode at the correct stage. It is a combinational circuit because anode activation depends directly on the counter value.

Implementation

The input to the module is the counter state, and its outputs are an0, an1, an2, and an3. It uses an always block sensitive to count (the counter state), containing a case statement that selects which anode is activated. As shown in the accompanying illustration, an3 is activated in state 1110, an2 in 1010, an1 in 0110, and an0 in 0010.

Τιμή μετρητή	AN3	AN2	AN1	AN0
1111	1	1	1	1
1110	**0**	1	1	1
1101	1	1	1	1
1100	1	1	1	1
1011	1	1	1	1
1010	1	**0**	1	1
1001	1	1	1	1
1000	1	1	1	1
0111	1	1	1	1
0110	1	1	**0**	1
0101	1	1	1	1
0100	1	1	1	1
0011	1	1	1	1
0010	1	1	1	**0**
0001	1	1	1	1
0000	1	1	1	1

Anode activation for selected counter states.

KathodeDriving module

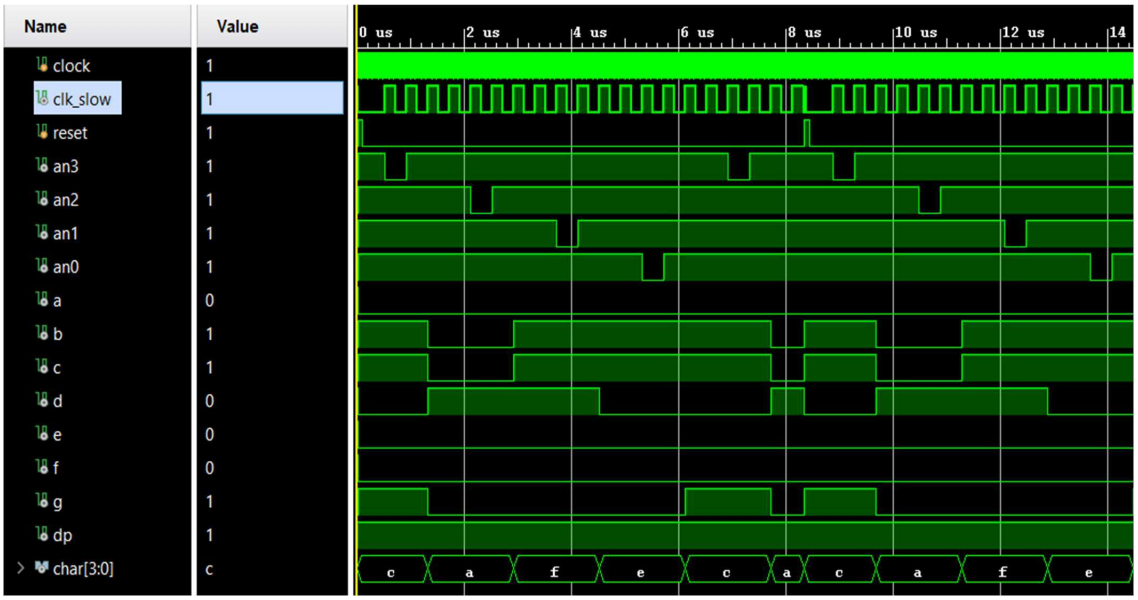
The cathode-driving module activates the cathodes for the appropriate interval, allowing the capacitors on the board to charge and discharge fully. It uses the counter state to select the desired values and therefore implements combinational logic.

Implementation

The module receives count (the counter state) and produces the 4-bit signal char, representing the hexadecimal character to be decoded. It contains an always block sensitive to count with a case statement covering all 16 states. A cathode must be activated two states before its corresponding anode and remain active until one state afterward. Specific values are assigned to char according to the laboratory requirements.

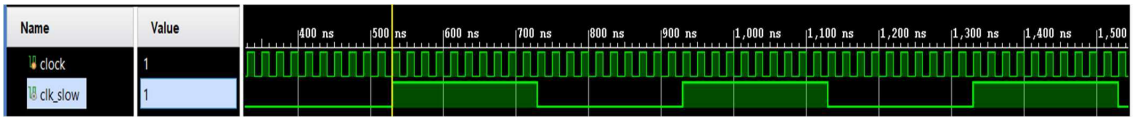
Verification

In the simulation testbench, clock and reset pulses are generated at regular intervals to provide sufficient samples for verifying the implementation. The resulting waveforms are examined to confirm that the signals take their expected values. Owing to the simplicity of the circuit, no problems were encountered during this test.



Simulated clock, reset, anode, cathode, and character signals.

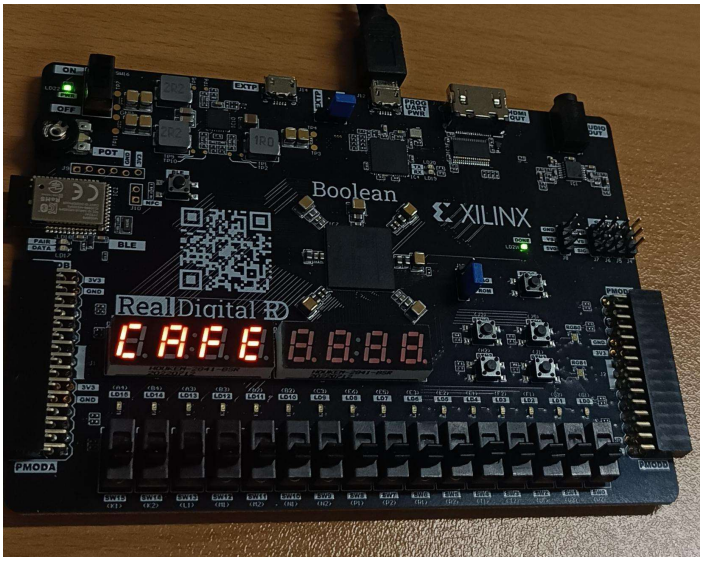
The anodes and cathodes can be seen to take the correct values at the appropriate instants. When reset is activated, the message is displayed again from the beginning.



Detailed view of the simulated clock signals.

Experiment and Final Verification

During FPGA board programming, no significant hardware-platform issues were encountered. The only problem arose in the counter implementation, where blocking assignments had been used in a sequential circuit. These were corrected to non-blocking assignments.

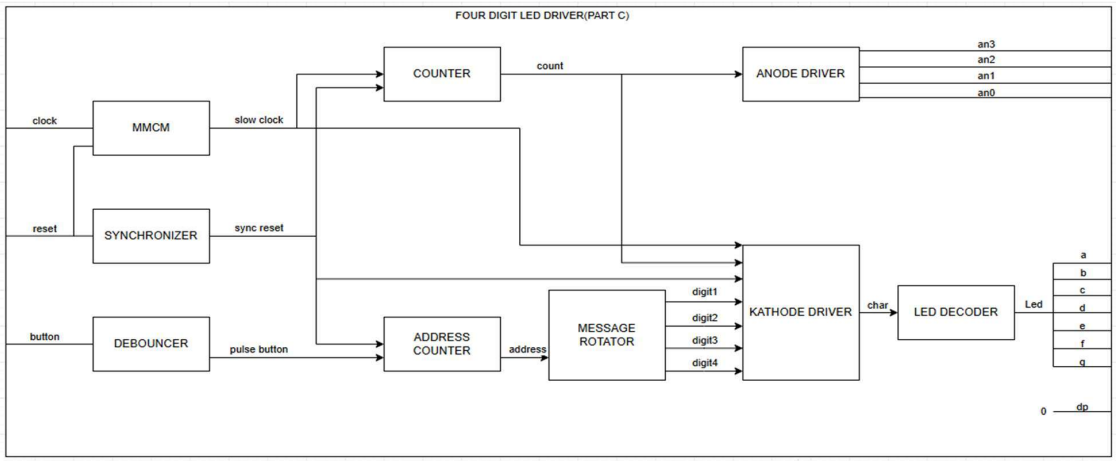


Four-digit LED driver operating on the FPGA board.

PART C: Stepwise Message Rotation Using a Pushbutton

Implementation

Top-level module overview



Architecture for pushbutton-controlled message rotation (Part C).

The module includes the following components:

- address
- LEDdecoder (Part A)
- AnodeDriving (Part B)
- KathodeDriving2
- reset_synchr
- button_deb
- MessageRotator

reset_synchr module

The reset synchronizer receives the asynchronous reset signal from the board and synchronizes it to the rising edge of the clock. The resulting circuit is sequential and requires two flip-flops.

Implementation

The module uses an always block with posedge clock and posedge reset in its sensitivity list. The inputs are clock and asynchronous reset, and the output is the synchronized reset signal. The flip-flops change the reset state from 1 to 0 in two stages: first from 11 to 10, and then to 00.

button_deb module

This module debounces the pushbutton on the FPGA board to generate a more stable signal when the button is pressed. The resulting pulse lasts for one clock period. The circuit is sequential.

Implementation

Its inputs are clock and the board pushbutton, and its output is the debounced button signal. The module contains three always blocks, each sensitive to posedge clock. The first uses two flip-flops to synchronize the pushbutton and establish the current value of synchr_button. The second checks whether this value remains stable for a sufficient interval by means of a counter; the required counter value differs between simulation and experimental testing on the FPGA board. The third ensures that the debounced button output is asserted for exactly one clock pulse.

KathodeDriving2 module

This module is an updated version of KathodeDriving from Part B. Changes were required to support the intended rotation of the displayed message. Specifically, its inputs now include both the counter value and the digits to be displayed at a given instant.

Implementation

An always block is used with count (counter state) and digit1, digit2, digit3, and digit4 (the four displayed characters) in its sensitivity list. A case statement selects the appropriate outputs based on the counter state.

MessageRotator module

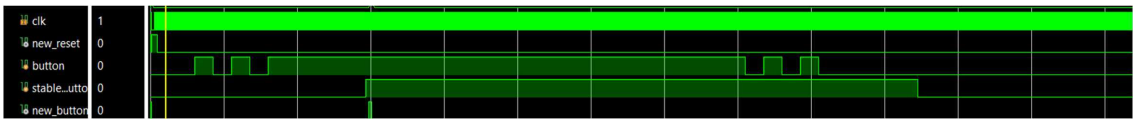
This module effectively implements a memory unit that stores the message to be rotated, consisting of 16 hexadecimal characters. Based on the supplied memory address, the appropriate values are assigned sequentially to the digits on each rising clock edge.

Implementation

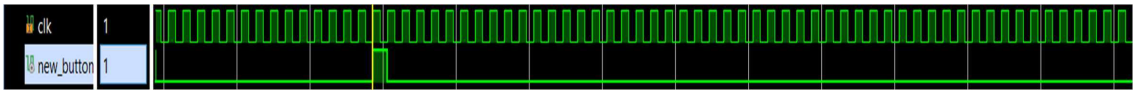
The inputs are clock, reset, and address, while the outputs are digit1, digit2, digit3, and digit4. Two always blocks sensitive to posedge clock are used: one to initialize the message and the other to assign the desired values to the four digit outputs.

Verification

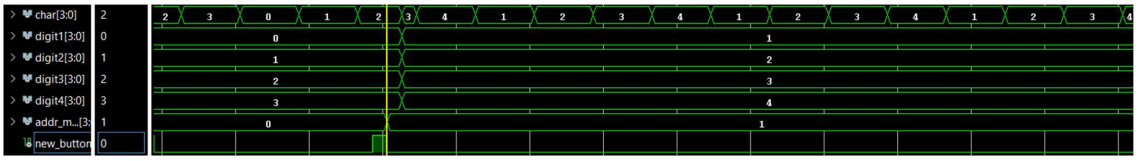
The simulation testbench defines the clock and reset pulses, as well as suitable pushbutton pulse durations, to verify the elimination of unstable button signals and the correct display of the message. Some timing problems were observed because, in the initial implementation, the duration of the debounced button pulse had not been constrained and therefore lasted several clock cycles.



Pushbutton and debouncing signals.



One-clock-period pulse generated by the debouncer.



Character and digit waveforms during message rotation.

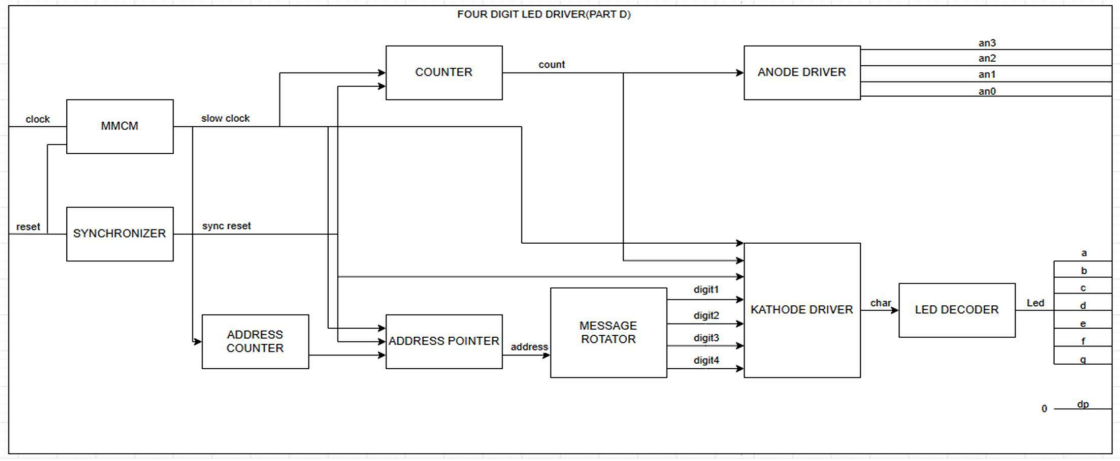
Experiment and Final Verification

During experimental testing on the FPGA board, the correct message was initially not displayed because of an incorrect counter calculation. The counter value was therefore recalculated to provide the appropriate interval for filtering unstable signals.

PART D: Stepwise Message Rotation with a Fixed Delay

Implementation

Top-level module overview



Architecture for automatic message rotation with a fixed delay (Part D).

This module includes the following components:

- counter_D
- LEDdecoder (Part A)
- AnodeDriving (Part B)
- KathodeDriving2 (Part C)
- reset_synchr (Part C)
- pointer
- MessageRotator (Part C)

counter_D module

This module increments the counter while reset is inactive. It is a sequential circuit.

Implementation

The inputs are clock and reset, and the output is counter. The implementation uses an always block sensitive to posedge clock, in which reset is checked so that the counter returns to its initial value whenever reset is active.

pointer module

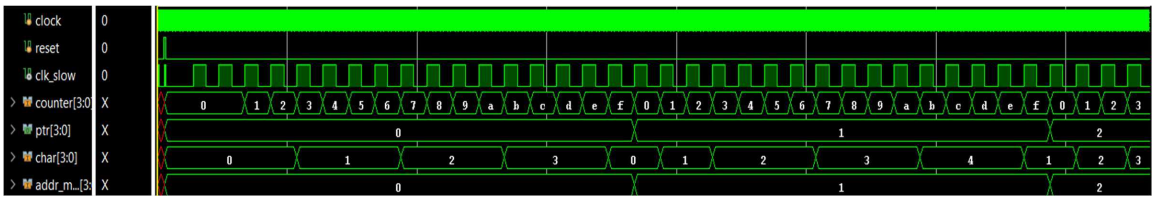
This sequential circuit increments the address by one when the counter reaches the appropriate value, that is, when an interval of 0.5 ms has elapsed. This interval is regarded as sufficient for a signal to be considered stable.

Implementation

The inputs are clock, reset, and counter, and the output is the address ptr. An always block sensitive to posedge clock checks reset and the counter value.

Verification

The simulation testbench sets the durations of clock and reset pulses so that the events can be verified as thoroughly as possible. The first simulated waveforms revealed timing mismatches in the message characters. These arose because the counter was both checked and assigned inside the same always block. Consequently, the logic was separated into two modules and the modules were connected in the top-level design.



Simulated counter, pointer, and character values during automatic rotation.

Experiment and Final Verification

No problems were observed during testing on the FPGA board, as the relevant issues had already been identified and corrected in earlier verification steps.

CONCLUSIONS: Summary of the Work

The assignment was completed successfully, meeting all its requirements and resolving the problems encountered during implementation. The main challenge was synchronizing the various signals in Parts C and D. Thorough analysis of the testbenches and waveforms, together with observation of the results on the FPGA board, helped identify the causes of these problems and resolve them in all affected modules.